

Bloque 01 - Fundamentos de datos: desde archivos hasta calidad

Propósito

Antes de programar o abrir una base de datos, Leo debe entender qué representa un dato, cómo se organiza la información y por qué una cifra aparentemente correcta puede conducir a una decisión equivocada. Este bloque no presupone que sepas qué es CSV, JSON, una tabla o una clave.

Resultado de salida

Al terminar podrás abrir un archivo sencillo y explicar qué información contiene, distinguir una tabla de un documento JSON, identificar el grano de un conjunto de datos, proponer controles de calidad y reconocer límites de privacidad y sesgo.

Lecciones

1. Qué es un dato, un archivo y una tabla.
2. Filas, columnas, tipos y relaciones.
3. CSV, JSON, Excel, Parquet y bases de datos.
4. Calidad, ausencia, sesgo, privacidad y ética.

Práctica

Realiza [la auditoría de calidad](#) después de la lección 4 y compárala con [la solución razonada](#).

01.1 Qué es un dato, un archivo y una tabla

Objetivos

Entender qué es un dato antes de hablar de herramientas; distinguir una información suelta de un conjunto organizado; y reconocer qué representa una fila de una tabla.

Empieza por una situación cotidiana

Imagina una tienda que quiere saber qué productos se venden más. Cada vez que una persona compra algo, la tienda puede guardar información: fecha, producto, importe y forma de pago. Cada una de esas piezas es un **dato**: una representación de algo que ocurrió en el mundo real.

La información se guarda en un **archivo**, igual que una fotografía o una nota de texto. Un archivo tiene nombre, contenido y un formato que indica cómo se organiza. No hay magia: un archivo de datos es una manera de guardar información para poder leerla, compartirla o analizarla después.

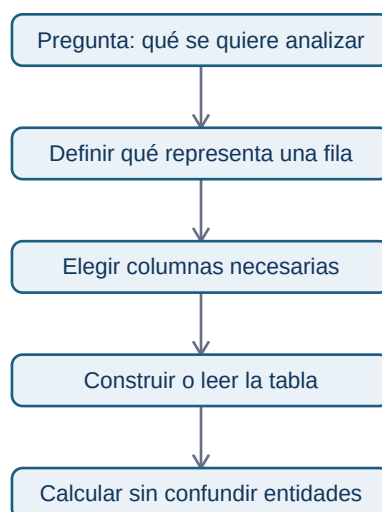
Una de las formas más comunes de organizar datos es una **tabla**. Una tabla se parece a una hoja de cálculo: tiene columnas que describen qué tipo de información guardamos y filas que recogen casos concretos.

fecha	producto	importe	canal
2026-01-03	teclado	45.99	web
2026-01-04	ratón	19.90	tienda
2026-01-04	teclado	45.99	web

En este ejemplo, la primera fila de datos representa una compra concreta. La columna **producto** responde qué se compró; **importe**, cuánto costó. La tabla no “sabe” qué significa un teclado: nosotros le damos significado a las columnas.

El grano: la pregunta que evita errores grandes

El **grano** indica qué representa exactamente una fila. Aquí, una fila representa una compra, no un cliente ni un producto. Esta frase parece pequeña, pero evita errores muy caros: si se cuenta una fila como si fuera una persona, un cliente que compra tres veces se contará como tres clientes.



Antes de cualquier análisis, completa siempre esta frase: “cada fila de este conjunto representa...”. Si no puedes completarla, no empieces a sumar ni a calcular promedios.

Ejemplo IT

Una aplicación registra eventos. Una tabla puede tener una fila por clic, otra por sesión y otra por usuario. Son tablas distintas con granos distintos. Contar clics no responde cuántos usuarios usaron la aplicación; contar sesiones tampoco responde directamente cuántas personas pagaron.

Error frecuente

Pensar que una tabla es “la realidad”. Una tabla es un modelo parcial. Puede no incluir compras hechas por teléfono, usuarios anónimos o devoluciones. Siempre pregunta qué no está en los datos.

Comprobación

Para una academia online, escribe el grano de tres tablas posibles: una de alumnos, una de inscripciones y una de visualizaciones de vídeo. ¿Por qué no deben mezclarse sin una relación explícita?

01.2 Filas, columnas, tipos y relaciones

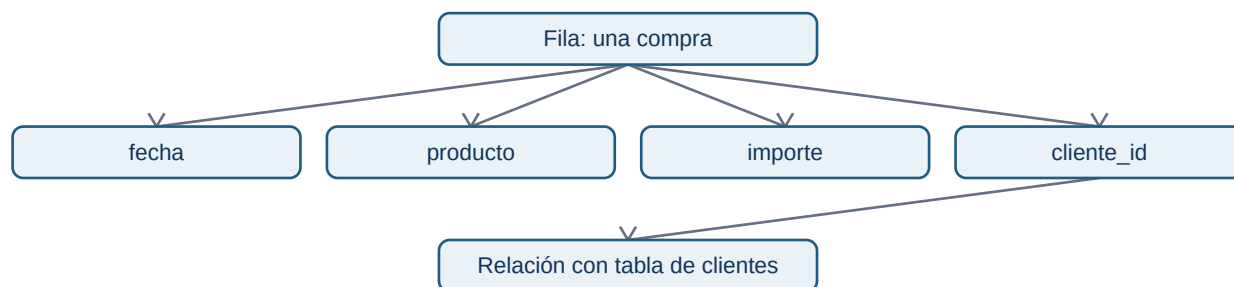
Objetivos

Aprender a leer una tabla con precisión: distinguir filas y columnas, reconocer tipos básicos de datos y entender por qué una clave permite relacionar tablas sin duplicar significado.

Filas y columnas no son solo una forma visual

Una **fila** contiene información de un caso. Una **columna** guarda el mismo tipo de atributo para muchos casos. En una tabla de compras, `importe` debería contener números; en una tabla de usuarios, `fecha_registro` debería contener fechas. El tipo de información determina qué operaciones tienen sentido.

No tiene sentido calcular la media de ciudad; sí puede tener sentido contar cuántos usuarios hay por ciudad. No tiene sentido ordenar alfabéticamente un importe para encontrar la venta mayor; sí tiene sentido convertirlo a número y compararlo.



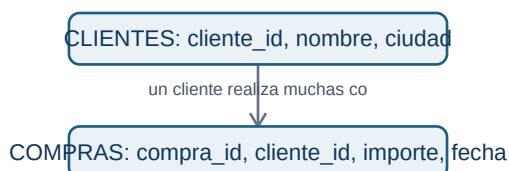
Tipos que encontrarás al empezar

- **Texto:** nombre de producto, ciudad, comentario.
- **Número:** importe, cantidad, edad, latencia.
- **Fecha y hora:** momento de registro, compra o despliegue.
- **Booleano:** verdadero/falso; por ejemplo, es_cliente.
- **Categoría:** conjunto limitado de etiquetas, como plan gratis, pro o empresa.

Un número puede representar cosas distintas: un identificador `cliente_id=1042` parece número, pero no debes calcular su media. Es una etiqueta técnica, no una cantidad.

Claves y relaciones

Una **clave** es una columna que permite identificar o conectar información. Si `cliente_id` identifica de forma única a cada cliente, se llama clave primaria de la tabla de clientes. La misma columna puede aparecer en la tabla de compras para indicar quién realizó cada compra; allí actúa como clave foránea o referencia.



La relación dice: un cliente puede realizar muchas compras; una compra pertenece a un cliente. Esta información es esencial al combinar tablas. Si una tabla de clientes tiene accidentalmente dos filas para el mismo `cliente_id`, una unión puede duplicar importes sin que el error sea evidente.

Error frecuente

Confundir un identificador con una medida. `pedido_id` y `cliente_id` sirven para identificar; no para hacer promedios. También es un error asumir que una columna “única” realmente lo es sin comprobar duplicados y nulos.

Comprobación

Diseña las columnas mínimas de una tabla de tickets de soporte. ¿Qué representa cada fila? ¿Qué columna conectaría un ticket con un cliente? ¿Cuál parece numérica pero no debe tratarse como medida?

01.3 CSV, JSON, Excel, Parquet y bases de datos

Objetivos

Saber qué problema resuelve cada formato básico y elegir una forma razonable de guardar o recibir información sin memorizar una lista de siglas.

CSV: una tabla escrita como texto

Un archivo **CSV** significa **Comma-Separated Values**: valores separados por comas. Es un archivo de texto que representa una tabla. La primera línea suele contener los nombres de las columnas; cada línea posterior es una fila.

```
fecha, producto, importe
2026-01-03, teclado, 45.99
2026-01-04, ratón, 19.90
```

CSV es sencillo de abrir, compartir y leer con Python, Excel o un editor de texto. Su simplicidad también tiene límites: no guarda bien tipos complejos, fórmulas, varias hojas ni una estructura dentro de otra. Además, hay que acordar separador, codificación, formato de fecha y separador decimal.

JSON: una ficha que puede contener otras fichas

JSON significa **JavaScript Object Notation**. Es texto estructurado mediante pares `campo: valor`. A diferencia de un CSV, puede guardar objetos dentro de objetos y listas. Es frecuente en APIs, configuraciones y eventos de aplicaciones.

```
{
  "pedido_id": 1001,
```

```

"cliente": {
  "nombre": "Leo",
  "ciudad": "Madrid"
},
"productos": ["teclado", "ratón"],
"importe": 65.89
}

```

El JSON anterior es una ficha de pedido. No es una tabla, aunque se pueda transformar en una. Para analizar muchos pedidos, normalmente tendrás que decidir qué campos extraer y cómo convertir listas u objetos anidados en columnas o tablas relacionadas.



Excel, Parquet y bases de datos

Excel es una aplicación y un formato de libro de trabajo; es útil para revisión manual, cálculos ligeros y comunicación. No es ideal como fuente única de procesos repetibles si múltiples personas lo editan sin control.

Parquet es un formato optimizado para datos tabulares grandes. Guarda columnas de forma eficiente y conserva tipos mejor que CSV. Normalmente lo usarás mediante Python, Spark, DuckDB o un warehouse, no editándolo a mano.

Una **base de datos** organiza información para que aplicaciones y personas puedan consultarla con reglas de acceso, relaciones y actualizaciones. SQL es un lenguaje para consultar muchas bases relacionales; MongoDB almacena documentos similares a JSON; DynamoDB se diseña alrededor de claves y patrones de acceso.

Cómo elegir sin obsesionarte

Empieza preguntando: ¿necesito una tabla simple que cualquiera pueda abrir? CSV. ¿Recibo una respuesta con estructuras anidadas de una API? JSON. ¿Trabajo con muchos datos tabulares repetidamente? Parquet o una base de datos. ¿Necesito revisar manualmente algo pequeño? Excel puede ser adecuado.

La elección no elimina el deber de conocer grano, calidad y significado.

Comprobación

Explica a otra persona la diferencia entre CSV y JSON sin usar las palabras “plano” ni “anidado”. Después indica cuál esperarías recibir de una API meteorológica y por qué.

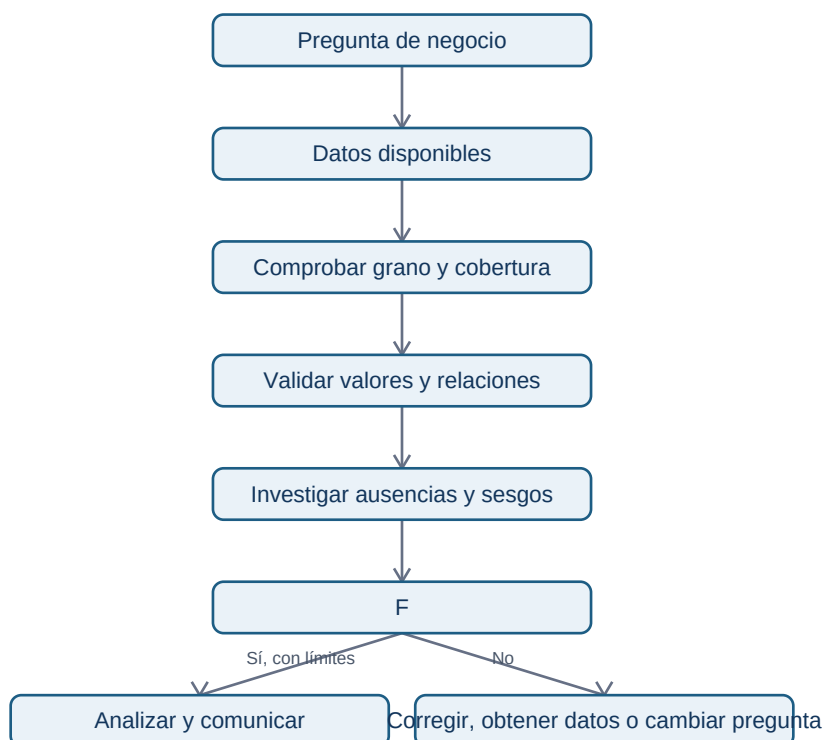
01.4 Calidad, ausencia, sesgo, privacidad y uso responsable

Objetivos

Revisar un conjunto de datos antes de analizarlo, interpretar ausencias sin borrarlas por costumbre y reconocer que una decisión basada en datos también puede causar daño.

Calidad como condición de confianza

La calidad no significa que un dataset sea perfecto. Significa que conocemos si es adecuado para una decisión concreta. Una tabla de compras puede ser suficiente para estimar ingresos diarios y no serlo para saber satisfacción de clientes.



Cinco controles iniciales son especialmente útiles:

- **Compleitud:** ¿faltan valores necesarios para la pregunta?
- **Validez:** ¿los valores respetan reglas, unidades y formatos?
- **Consistencia:** ¿la misma idea está registrada de la misma forma?
- **Unicidad:** ¿existen duplicados indebidos?
- **Actualidad:** ¿el dato llega a tiempo para la decisión?

Los nulos cuentan una historia

Un valor ausente no es automáticamente un error. Puede significar “no se aplica”, “no se midió”, “falló el sistema” o “la persona prefirió no responder”. Borrar todas las filas con nulos puede eliminar justo a la población con la que tienes un problema.

Por ejemplo, si el campo ingresos falta sobre todo en usuarios que abandonan un formulario, la ausencia es información sobre fricción. Antes de imputar o eliminar, mide dónde faltan datos, desde cuándo y en qué segmentos.

Sesgo, privacidad y propósito

Un dataset puede representar peor a grupos que usan menos una aplicación, tienen conectividad limitada o no están incluidos en la fuente. Un modelo entrenado con esos datos puede amplificar esa desigualdad. El analista debe declarar cobertura, exclusiones y riesgos, no tratarlos como una nota al pie.

La privacidad comienza antes de abrir un archivo: recoge solo los datos necesarios, evita copiar identificadores personales en notebooks, limita acceso y define cuánto tiempo se conservan. Que un sistema permita acceder a una columna no significa que sea legítimo usarla para cualquier objetivo.

Caso práctico

Una empresa quiere comparar uso por ciudad, pero el 30 % de usuarios no informa ciudad y ese porcentaje es mayor en móvil. Concluir que “móvil usa menos el producto en ciertas ciudades” sin estudiar la ausencia puede ser falso. Primero se investiga el formulario, la geolocalización, el consentimiento y los segmentos afectados.

Comprobación

Elige una de las cinco dimensiones de calidad y describe: un error concreto, cómo lo detectarías, qué decisión podría dañar y cuál sería una respuesta prudente.